

VLLM专题（三十九）—自动前缀缓存（二）

AI专题精讲 已于 2025-03-19 10:08:46 修改 阅读量490 收藏 点赞数 CC 4.0 BY-SA版权

分类专栏: 大模型专题系列 文章标签: 缓存 哈希算法 算法



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容 查看更多超级会员权益

前缀缓存（Prefix Caching）是一种在LLM推理中广泛使用的优化技术，旨在避免冗余的提示词（prompt）计算。其核心思想很简单——我们缓存已处理请求的键值缓存（kv-cache）块，并在新请求的前缀与之前请求相同时重用这些块。由于前缀缓存几乎是一种“免费的午餐”，并且不会改变模型输出，因此它已被许多公共端点（例如OpenAI、Anthropic等）和大多数开源LLM推理框架（例如SGLang）广泛采用。

尽管实现前缀缓存的方法有很多，但vLLM选择了一种基于哈希的方法。具体来说，我们通过对每个kv-cache块中的token以及该块之前的prefix中的token进行哈希来标识缓存块。

```

Block 1      Block 2      Block 3
[A gentle breeze stirred] [the leaves as children] [laughed in the distance]
Block 1: |<--- block tokens ---->|
Block 2: |<----- prefix ----->| |<--- block tokens ---->|
Block 3: |<----- prefix ----->| |<--- block tokens ---->|
```

在上面的示例中，第一个块的KV缓存可以通过token“A gentle breeze stirred”唯一标识。第三个块可以通过块中的token“laughed in the distance”以及前缀token“A gentle breeze stirred the leaves as children”唯一标识。因此，我们可以构建块的哈希值 `hash(tuple[components])`，其中 `components` 包括：

1. **父哈希值**：父哈希块的哈希值。
2. **块中的token**：该块中的token元组。包含确切token的原因是为了减少潜在的哈希值冲突。
3. **额外哈希值**：使该块唯一所需的其他值，例如LoRA ID和多模态输入哈希值（见下面的示例）。

注意 1：我们只缓存完整的块。

注意 2：上述哈希键结构并非100%无冲突。理论上，不同的前缀token仍有可能具有相同的哈希值，但这几乎不可能发生。当然，如果您有绝妙的想法可以完全消除冲突，欢迎贡献代码。

多模态输入的哈希示例

在这个示例中，我们展示了前缀缓存在多模态输入（例如图像）情况下的工作原理。假设我们有一个包含以下消息的请求：

```
messages = [
  {
    "role": "user",
    "content": [
      {
        "type": "text",
        "text": "What's in this image?"
      },
      {
        "type": "image_url",
        "image_url": {
          "url": image_url,
        },
      },
    ],
  },
]
```

它将生成以下提示词（prompt）：

Prompt:

```
<s>[INST]What's in this image?\n[IMG][/INST]
```

Tokenized prompt:

```
[1, 3, 7493, 1681, 1294, 1593, 3937, 9551, 10, 4]
```

Prompt with placeholders (<P>):

```
[1, 3, 7493, 1681, 1294, 1593, 3937, 9551, <P>, <P>, ..., <P>, 4]
```

如我们所见，分词后，`[IMG]` 将被一系列占位符token替换，这些占位符将在预填充（prefill）阶段被图像嵌入（image embeddings）替换。前缀缓存在支持这种情况时面临的挑战是，我们需要将图像与占位符区分开来。为了解决这个问题，我们对前端图像处理器生成的图像哈希进行编码。例如，上述提示词中块的哈希值将如下所示（假设块大小为16，并且我们有41个占位符token）：

```
Block 0
  Parent hash: None
  Token IDs: 1, 3, 7493, 1681, 1294, 1593, 3937, 9551, <p>, ..., <p>
  Extra hash: <image hash>
Block 1
  Parent hash: Block 0 hash
  Token IDs: <p>, ..., <p>
  Extra hash: <image hash>
Block 2
  Parent hash: Block 1 hash
  Token IDs: <p>, ..., <p>
  Extra hash: <image hash>
Block 3
  Parent hash: Block 2 hash
  Token IDs: <p>, ..., <p>, 4
  Extra hash: <image hash>
```

在本文件的其余部分，我们首先介绍 vLLM v1 中用于前缀缓存的数据结构，然后介绍主要 KV 缓存操作符（例如：分配、追加、释放、驱逐）的前缀缓存工作流。最后，我们通过一个示例来说明端到端的前缀缓存工作流。

1. 数据结构

vLLM v1 中的前缀缓存是在 KV 缓存管理器中实现的。基本构建块是“块”（Block）数据类（简化版）：

```
class KVCacheBlock:
    # 块的 ID (不可变)
    block_id: int
    # 块的哈希 (当块满时分配, 且当块被驱逐时重置)
    block_hash: BlockHashType
    # 当前使用此块的请求数量
    ref_cnt: int

    # 双向链表的指针, 用于形成空闲队列
    prev_free_block: Optional["KVCacheBlock"] = None
    next_free_block: Optional["KVCacheBlock"] = None
```

有两个设计要点需要强调:

我们在初始化 KV 缓存管理器时分配所有 KVCacheBlock 作为一个块池。这避免了 Python 对象创建的开销, 并且能够轻松追踪所有块的状态。

我们在 KVCacheBlock 中直接引入了双向链表指针, 以便直接构建一个空闲队列。这给我们带来了两个好处:

- 我们可以在 $O(1)$ 时间复杂度下将中间元素移动到队尾。
- 我们可以避免引入另一个 Python 队列 (例如: deque), 从而避免对元素进行包装。

因此，当 KV 缓存管理器初始化时，我们将拥有以下组件：

块池（Block Pool）：一个存储 `KVCacheBlock` 的列表。

空闲块队列（Free Block Queue）：仅存储头块和尾块的指针，用于操作。

缓存块（Cache blocks）：从哈希键到块ID的映射。

请求块（Request blocks）：从请求ID到已分配块ID的映射。

2. 操作

块分配（Block Allocation）

新请求：调度器调度新请求并分配KV缓存块的工作流程：

1. 调度器调用 `kv_cache_manager.get_computed_blocks()`，获取已经计算过的块序列。这是通过对请求中的提示词（prompt tokens）进行哈希并在缓存块（Cache Blocks）中查找来完成的。
2. 调度器调用 `kv_cache_manager.allocate_slots()`，执行以下步骤：
 - 计算需要的新块数量，如果没有足够的块可供分配，则返回。
 - “触碰”（Touch）已计算的块。它将已计算块的引用计数加一，如果该块未被其他请求使用，则将其从空闲队列中移除。这是为了避免这些已计算的块被驱逐。详见下一节的示例说明。
 - 通过弹出空闲队列的头部来分配新块。如果头部块是缓存块，则同时“驱逐”该块，以确保其他请求无法再重用该块。
 - 如果分配的块已经填满token，则立即将其添加到缓存块（Cache Block）中，以便同一批次中的其他请求可以重用该块。

运行中的请求：调度器调度运行中的请求并分配KV缓存块的工作流程：

1. 调度器调用 `kv_cache_manager.append_slots()`，执行以下步骤：

- 计算需要的新块数量，如果没有足够的块可供分配，则返回。
- 通过弹出空闲队列的头部来分配新块。如果头部块是缓存块，则同时“驱逐”该块，以确保其他请求无法再重用该块。
- 将token ID追加到现有块和新块的槽中。如果块已满，则将其添加到缓存块（Cache Block）中以缓存它。

**** 重复块（Duplicated Blocks） ****

假设块大小为4，您发送一个请求（请求1），提示词为 `ABCDEF`，解码长度为3：

```
Prompt: [A, B, C, D, E, F]
Output: [G, H, I]

Time 0:
  Tokens: [A, B, C, D, E, F, G]
  Block Table: [0 (ABCD), 1 (EFG)]
  Cache Blocks: 0

Time 1:
  Tokens: [A, B, C, D, E, F, G, H]
  Block Table: [0 (ABCD), 1 (EFGH)]
  Cache Blocks: 0, 1

Time 2:
  Tokens: [A, B, C, D, E, F, G, H, I]
  Block Table: [0 (ABCD), 1 (EFGH), 2 (I)]
  Cache Blocks: 0, 1
```

此时，块0和块1已被缓存。

现在我们再次发送相同的请求（请求2），并使用贪婪采样（greedy sampling），因此它将生成与请求1完全相同的输出：

```
Prompt: [A, B, C, D, E, F]
Output: [G, H, I]

Time 0:
  Tokens: [A, B, C, D, E, F, G]
  Block Table: [0 (ABCD), 3 (EFG)]
  Cache Blocks: 0, 1
Time 1:
  Tokens: [A, B, C, D, E, F, G, H]
  Block Table: [0 (ABCD), 3 (EFGH)]
  Cache Blocks: 0, 1, 3
```

可以看到，块3是一个新的已满块，并被缓存。然而，它与块1重复，这意味着我们缓存了相同的块两次。在v0中，当检测到块3重复时，我们会释放块3，并让请求2改用块1，因此在时间1时，其块表变为 `[0, 1]`。然而，在vLLM v1中，块表是仅追加的（append-only），这意味着将块表从 `[0, 3]` 更改为 `[0, 1]` 是不允许的。因此，我们将为哈希键 `E-H` 保留重复的块。这种重复将在请求释放时被消除。

2.1 释放（Free）

当一个请求完成时，如果没有其他请求正在使用其块（引用计数为0），我们将释放该请求的所有块。在这个示例中，我们释放了请求1及其关联的块2、3、4、8。可以看到，释放的块以相反的顺序被添加到空闲队列的尾部。这是因为请求的最后一个块哈希了更多的token，因此被其他请求重用的可能性较低。因此，它应该首先被驱逐。

2.2 驱逐（LRU）

当空闲队列的头部块（最近最少使用的块）被缓存时，我们必须驱逐该块，以防止其他请求使用它。具体来说，驱逐包括以下步骤：

1. 从空闲队列的头部弹出该块。这是要驱逐的LRU块。
2. 从缓存块（Cache Block）中移除该块的ID。
3. 移除该块的哈希值。

3. 示例

在这个示例中，我们假设块大小为4（每个块可以缓存4个token），并且KV缓存管理器中共有10个块。

时间1：缓存为空，一个新的请求到来。我们分配了4个块。其中3个块已经填满并被缓存。第四个块部分填充，4个token中填充了2个。



AI专题精讲

已关注

0

0

0

分享

专栏目录

已订阅